Lesson 4: Advanced Git Workflows, Undoing Mistakes & Best Practices

In Lesson 3, you connected Git to GitHub and mastered the Pull Request process. In this final lesson, you will learn essential diagnostic tools, safety nets for fixing common mistakes, and industry best practices for keeping your commit history clean.

1. Undoing Mistakes Safely

Every developer makes mistakes. Git provides distinct commands depending on *where* the mistake happened and *how* you want to fix it.

A. Unstaging Files (git restore)

If you staged a file by accident using git add, but haven't committed yet:

Bash

```
# Unstage a specific file without losing your changes

git restore --staged filename.txt
```

B. Discarding Uncommitted Changes

If you made mess-up edits in your working directory and want to revert to the last commit:

Bash

```
# Danger: Discards all uncommitted local edits in this file

git restore filename.txt
```

C. Safely Reverting a Published Commit (git revert)

If a commit was already pushed to GitHub and broke something, do not erase history. Use git revert to create a *new* commit that undoes the changes.

Bash

```
# Reverts the specified commit safely without modifying past history

git revert <commit-hash>
```

2. Viewing History & Debugging (git log & git blame)

When tracking down when a bug was introduced or who wrote a specific line of code, use Git's built-in inspection tools.

Modern Log Filtering

Bash

```
# Compact single-line view showing branch connections visually

git log --oneline --graph --all
```

 # Search commits by message keyword

git log --grep="login bug"

Line-by-Line Code Attribution (git blame)

Find out who modified each line of a file and in which commit:

Bash

git blame filename.txt

3. Temporarily Saving Work (git stash)

Imagine you are mid-way through building a feature on feature-ui, and an urgent hotfix is needed on main. You aren't ready to commit your unfinished work.

Bash

 # 1. Save uncommitted changes to a temporary shelf

git stash

 # 2. Switch branches and complete the hotfix

git switch main

 # ... fix bug and commit ...

 # 3. Return to your feature branch and restore stashed work

git switch feature-ui

git stash pop

4. Professional Git Best Practices

Following standardized team protocols keeps project histories clean and easy to maintain.

Practice	Recommendation	Anti-Pattern
Commit Size	Small, atomic commits covering one logical change.	Massive commits changing 20 unrelated files.
Commit	Clear, imperative tense (Add user auth, Fix nav	Vague messages (fix bug, update, asdf).

Practice	Recommendation	Anti-Pattern
Messages	layout).	
Branch Naming	Descriptive prefixes (feature/login, fix/navbar-overflow).	Generic names (test, branch1, my-work).
Pulling Changes	Pull/rebase frequently before opening PRs.	Waiting 2 weeks to sync with main.

💡 **Practical Exercise**

Practice using safety nets in your terminal:

1. **Edit notes.txt with some random text and stage it using git add notes.txt.**
2. **Unstage it using git restore --staged notes.txt and verify with git status.**
3. **Make another quick edit to notes.txt, then run git stash. Run git status to confirm your working directory is clean.**
4. **Retrieve your changes back using git stash pop.**
5. **Run git log --oneline to locate a previous commit hash, then test git blame notes.txt to inspect line authors.**